UNIVERSIDAD CATÓLICA DE EL SALVADOR

Facultad de Ingeniería y Arquitectura

MEMORIA TÉCNICA DE AVANCE — LABORATORIO

Sistema de Gestión de Mantenimiento Preventivo de Equipos Médicos e Industriales (MedControl)

Asignatura:	Cómputo en la Nube — Grupo B
Catedrático:	Ing. Herberth Contreras
Fecha de Entrega:	Sábado 7 de Noviembre de 2026
Estudiante:	**Douglas Moran**
Rol Asignado:	**Desarrollador 2 (Backend Maintenance API)**
Responsabilidad:	Lógica para generación de órdenes de trabajo y cálculo de fechas de mantenimiento preventivo
Repositorio GitHub:	`https://github.com/jeffreyM1278/Proyecto.git`
Rama Evaluada:	`feature/backend-maintenance-api`

1. RESUMEN EJECUTIVO Y MATRIZ DE EVALUACIÓN

El presente informe acredita el cumplimiento exhaustivo del 100% de los aspectos requeridos para el **Primer Avance de Laboratorio** según la rúbrica oficial establecida por la cátedra. Como Desarrollador 2, se implementó el motor lógico del sistema **MedControl** mediante una API REST en Node.js y Express con arquitectura en capas limpias, soporte dual para SQL Server y MySQL, almacenamiento en la nube con Amazon S3 y autorización granular basada en roles (RBAC).

Criterio de Evaluación	Ponderación	Estado	Evidencia Presentada
1. Repositorio, Git y Estructura	25%	**Cumplido**	Carpeta modular `/backend`, `.gitignore` estricto, rama `feature/backend-maintenance-api` con 10 commits trazables.
2. Backend, API y Base de Datos	25%	**Cumplido**	Script Sequelize multicompatible (SQL Server / MySQL), 9 módulos REST y 2 CRUDs completos probados con Postman.
3. Lógica Preventiva (Rol Dev 2)	Diferenciador	**Cumplido**	Motor de cálculo de fechas (días, meses, años) y generación automática de órdenes sin duplicidad.
4. Infraestructura AWS y Seguridad	25%	**Cumplido**	Amazon S3 con URLs prefirmadas temporales, control IAM/RBAC (solo ADMIN borra) y gestión segura con `.env`.

Criterio de Evaluación	Ponderación	Estado	Evidencia Presentada
5. Alineación con Frontend (UI)	25%	Cumplido	Endpoints adaptados a los campos exactos del diseño MedControl en React + Vite + Tailwind CSS.

2. CRITERIO 1: REPOSITORIO, GIT Y ESTRUCTURA DE CÓDIGO (25%)

El backend se estructuró bajo el principio de separación de responsabilidades en el directorio `/backend`, desacoplando controladores, servicios de negocio, modelos relacionales y validaciones de entrada.

2.1. Historial de Commits en GitHub

Para evidenciar un avance progresivo y ordenado, el desarrollo se versionó en **10 commits consecutivos** en la rama `feature/backend-maintenance-api` siguiendo el estándar internacional *Conventional Commits*:

```
670bf69 (HEAD) feat(app): configure Express application entry point an
3f2fed3 feat(controllers-routes): implement REST endpoints and route h
0b7fd38 feat(services): implement AWS S3 document management and dashb
8c80271 feat(services): add domain services for equipment, technical h
c7c594a feat(services): implement core maintenance calculation and pre
52a638b feat(validators): create Joi request validation schemas for al
61a9153 feat(middlewares): add JWT authentication, RBAC authorization,
611d57a feat(models): implement Sequelize models matching relational d
8d78b97 feat(config): setup core configurations for database, AWS S3,
06877e1 chore(backend): initialize Node.js project and environment con
```

2.2. Seguridad en Git

El archivo `.gitignore` garantiza que no se suban dependencias (`node_modules/`), archivos de entorno con claves reales (`.env`), logs ni archivos del sistema operativo o

IDEs. Se provee `.env.example` como plantilla de configuración.

3. CRITERIO 2: AVANCE DE BACKEND, API Y BASE DE DATOS (25%)

3.1. Script de Conexión a Base de Datos (SQL Server / MySQL)

El archivo `src/config/database.js` implementa Sequelize con *Connection Pooling* (máximo 10 conexiones simultáneas), soporte para zona horaria de El Salvador (CST `-06:00`) y dialecto dinámico configurable por variable de entorno para adaptarse al motor seleccionado por el equipo:

```
import { Sequelize } from 'sequelize';
import config from './index.js';

const dialect = config.db.dialect || 'mssql'; // Soporta SQL Server (d

const sequelize = new Sequelize(
  config.db.name,
  config.db.user,
  config.db.password,
  {
    host: config.db.host,
    port: config.db.port,
    dialect,
    pool: { max: 10, min: 0, acquire: 30000, idle: 10000 },
    logging: config.app.env === 'development' ? console.log : false,
    dialectOptions: dialect === 'mssql' ? {
      options: { encrypt: false, trustServerCertificate: true, enableA
    } : {}
  }
);

export default sequelize;
```

3.2. Pruebas de Operaciones CRUD Reales (Postman)

CRUD 1: Inventario de Maquinaria (`/api/v1/equipment`)

Operación	Método y Ruta	Payload de Ejemplo / Descripción	Respuesta HTTP
Crear	`POST /api/v1/equipment`	Registra nuevo equipo con número de serie, modelo, marca, sede y frecuencia en días.	**201 Created**
Listar	`GET /api/v1/equipment?page=1&limit=10`	Retorna parque instalado con búsqueda por serie/modelo, filtros de estado y paginación.	**200 OK**
Actualizar	`PUT /api/v1/equipment/:id`	Actualiza estado operativo (`OPERATIVO` , `EN_MANTENIMIENTO` , `FUERA_DE_SERVICIO`).	**200 OK**
Eliminar	`DELETE /api/v1/equipment/:id`	Da de baja un equipo. Protegido: **Solo rol ADMIN**.	**200 OK / 403 Forbidden**

CRUD 2: Órdenes de Mantenimiento (`/api/v1/work-orders`)

Operación	Método y Ruta	Payload de Ejemplo / Descripción	Respuesta HTTP
Crear	`POST /api/v1/work-orders`	Crea orden Preventiva o Correctiva con técnico asignado, fecha y checklist en JSON.	**201 Created**
Listar	`GET /api/v1/work-orders`	Filtra órdenes por estado (`PENDIENTE` , `EN_PROGRESO` , `COMPLETADA`).	**200 OK**

Operación	Método y Ruta	Payload de Ejemplo / Descripción	Respuesta HTTP
Actualizar	`PUT /api/v1/work-orders/:id`	Registra avance, diagnóstico técnico y repuestos utilizados.	`200 OK`
Cerrar	`PATCH /api/v1/work-orders/:id/close`	Cierra la orden, sella la fecha de término y crea el asiento en bitácora técnica.	`200 OK`

4. LÓGICA CORE DE NEGOCIO (VALOR AGREGADO DEL DESARROLLADOR 2)

El aporte diferencial del rol de Douglas Moran consiste en la automatización del mantenimiento preventivo mediante dos algoritmos implementados en `src/services/maintenanceService.js`:

4.1. Algoritmo de Cálculo de Fechas Preventivas

Procesa la última fecha de mantenimiento y la frecuencia establecida en el plan del equipo:

Fórmula:

`FechaPróxima = FechaÚltimoMantenimiento + (FrecuenciaValor × FrecuenciaUnidad)`

`DíasRestantes = FechaPróxima - FechaActual`

- **VENCIDO (< 0 días):** Genera alerta crítica roja. Requiere intervención inmediata.
- **PRÓXIMO (0 a 7 días):** Genera alerta preventiva ámbar. Candidato para orden de trabajo.
- **AL DÍA (> 7 días):** Equipo en estado conforme verde.

4.2. Generador Automático de Órdenes sin Duplicados

Invocado mediante `POST /api/v1/maintenance/generate-orders`:

1. Identifica todos los equipos activos con estado **VENCIDO** o **PRÓXIMO** (≤ 7 días).
2. Verifica en `OrdenesMantenimiento` que **NO exista** ya una orden preventiva abierta (en estado `PENDIENTE` o `EN_PROGRESO`) para dicho equipo.
3. Si no existe orden previa, crea la orden preventiva con prioridad calculada (`ALTA` si está vencida, `MEDIA` si es próxima) y fecha programada.
4. Retorna un reporte detallando cuántas órdenes se crearon y cuántas se omitieron por ya estar en trámite.

5. CRITERIO 3: INFRAESTRUCTURA AWS CLOUD Y SEGURIDAD INICIAL (25%)

5.1. Almacenamiento en Amazon S3 con URLs Prefirmadas

Los manuales técnicos y fichas en PDF no se almacenan como binarios en la base de datos para no degradar el rendimiento relacional. Se utiliza el bucket de Amazon S3:

- **Carga:** Se sube mediante `PutObjectCommand` del SDK oficial `@aws-sdk/client-s3`.
- **Descarga Segura:** El bucket se mantiene totalmente privado. Para visualizar o descargar un documento, el endpoint `GET /api/v1/documents/:id/download` genera una **URL prefirmada temporal** (con expiración de 15 minutos) usando `@aws-sdk/s3-request-presigner`.

5.2. Control de Acceso Basado en Roles (AWS IAM / RBAC)

Se configuró un middleware de autorización que restringe las acciones según el rol del usuario:

- `ADMIN`: Acceso total. **Único perfil autorizado para operaciones** `DELETE`.
- `SUPERVISOR`: Lectura total, creación y actualización de órdenes y programación.
- `TECNICO`: Visualización de órdenes asignadas, actualización de diagnósticos y bitácora técnica.

6. RECURSOS Y ARCHIVOS ENTREGABLES

1. **Colección de Postman:** Archivo `MedControl-API.postman_collection.json` ubicado en la raíz de `/backend`, listo para importar.
2. **Documentación Swagger UI:** Disponible de forma interactiva en `http://localhost:3000/api-docs`.
3. **Código Fuente en GitHub:** Rama oficial `feature/backend-maintenance-api` en el repositorio `https://github.com/jeffreyM1278/Proyecto.git`.

Universidad Católica de El Salvador (UNICAES) — Cómputo en la Nube Grupo B — Ing. Herberth Contreras
Memoria Técnica Desarrollador 2: Douglas Moran — Sábado 7 de Noviembre de 2026